LP-III · FINAL YEAR B.E. · PRACTICALS 8 – 12

Machine Learning

Every practical explained from the ground up, with worked examples and probable viva / oral questions with answers

+	**Machine learning basics**	Before you start
8	**Uber fare prediction — Linear Regression and Random Forest**	Practical 8
9	**Email spam detection — KNN and SVM**	Practical 9
10	**Bank customer churn — Neural network classifier**	Practical 10
11	**K-Nearest Neighbours on the diabetes dataset**	Practical 11
12	**K-Means and Hierarchical clustering — Mall customers**	Practical 12
+	**Quick revision and cross-practical questions**	Revision

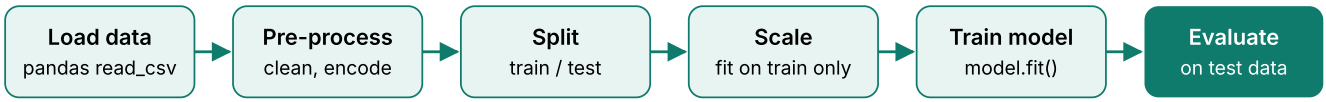
Based on the code in **github.com/qwerty7733/lp3**.

How to use: read the theory once, trace the worked example by hand, then cover the answers and test yourself on the questions. Answers are written the way you would say them in a viva — short first line, then the detail if the examiner digs deeper.

BEFORE YOU START

Machine learning basics

All five ML practicals follow the same pipeline. Examiners usually test this pipeline first, so understand it before the individual algorithms.



The common workflow used in Practicals 8 – 12.

Types of machine learning

Type	Data	Goal	Your practicals
Supervised	features X **with** labels y	learn a mapping X → y	8 (regression), 9, 10, 11 (classification)
Unsupervised	features only, no labels	find structure / groups	12 (clustering)
Reinforcement	rewards from an environment	learn actions that maximise reward	—

Within supervised learning, **regression** predicts a continuous number (Uber fare) and **classification** predicts a category (spam / not spam, leaves / stays, diabetic / not).

Key ideas used everywhere

- **Features (X)** are the inputs; the **target (y)** is what we predict.
- **Train/test split** — the model learns from the training set and is scored on the unseen test set. All practicals use 80/20 (`test_size=0.2`).
- `random_state` fixes the random shuffle so results are repeatable.
- `stratify=y` keeps the class ratio (e.g. 20% churners) the same in train and test.
- **Feature scaling** — distance-based (KNN, SVM, K-Means) and gradient-based (neural networks) models need features on similar scales.
- *Standardisation* (`StandardScaler`): z = (x − mean) / std → mean 0, std 1.
- *Min-max normalisation*: (x − min) / (max − min) → range 0 to 1.
- *Row normalisation* (`normalize`, Practical 9): each row is scaled to length 1.
- **Data leakage** — any information from the test set used during training (e.g. computing the scaler's mean on the full data). It gives falsely high scores. That is why the code calls `fit_transform` on train and only `transform` on test.
- **Overfitting** — the model memorises training data including noise: high train score, low test score. **Underfitting** — the model is too simple: low on both.
- **Bias–variance trade-off** — simple models have high bias (underfit), complex models have high variance (overfit). The goal is the balance point.
- **Cross-validation** — split training data into k folds, train on k − 1 and validate on the remaining one, rotating k times; average the scores. Used with `GridSearchCV` in Practical 11.
- **Hyperparameter** — a setting chosen before training (k in KNN, C in SVM, number of trees, epochs). **Parameter** — learned from data (regression coefficients, network weights).

Warm-up questions

Q1. What is machine learning?

A field of AI where a program learns patterns from data and improves at a task through experience, instead of being explicitly programmed with rules.

Q2. Difference between supervised and unsupervised learning?

Supervised learning uses labelled data (each example has the correct answer) to learn to predict labels. Unsupervised learning has no labels and finds hidden structure such as clusters.

Q3. Difference between regression and classification?

Regression predicts a continuous value (fare = ₹245.6). Classification predicts a discrete class (spam or not spam).

Q4. Why do we split data into training and testing sets?

To measure how well the model generalises to data it has never seen. Scoring on training data would reward memorisation.

Q5. What is overfitting and how can you reduce it?

The model fits noise in the training data and performs poorly on new data. Remedies: more data, simpler model, regularisation (L1/L2, dropout), early stopping, cross-validation, pruning trees / limiting depth.

Q6. What is the difference between a parameter and a hyperparameter?

Parameters are learned during training (weights, coefficients). Hyperparameters are set by us before training (learning rate, k, number of layers) and are tuned with validation data.

Q7. Why is feature scaling needed?

Features with large ranges (salary in lakhs) dominate distances and gradients over features with small ranges (age). Scaling puts them on equal footing; it is essential for KNN, SVM, K-Means and neural networks but not for tree-based models.

Q8. What is data leakage?

When information that would not be available at prediction time (usually from the test set) is used during training — for example scaling with the mean of the whole dataset. It produces over-optimistic evaluation.

Q9. What libraries did you use?

pandas (load and clean tables), NumPy (maths), scikit-learn (splitting, scaling, models, metrics), TensorFlow/Keras (neural network), matplotlib (plots) and SciPy (dendrogram).

PRACTICAL 8

Uber fare prediction — Linear Regression and Random Forest

Aim. Predict the price of an Uber ride from a pickup point to a drop-off point: (1) pre-process the dataset, (2) identify outliers, (3) check correlation, (4) implement linear regression and random forest regression, (5) evaluate and compare with R^2, RMSE, etc.

Code: `ML_UberFarePrediction_pract8.py`. **Dataset:** `uber.csv` (about 200,000 rides) with `fare_amount`, `pickup_datetime`, pickup/drop-off latitude and longitude, and `passenger_count`.

Step 1 — Pre-processing

Step in the code	Why
Drop `Unnamed: 0` and `key`	they are just row IDs; no relation to the fare
`dropna()`	a few rows have missing drop-off coordinates
Convert `pickup_datetime` and extract **hour, weekday, month, year**	a model cannot use a text date; but rush hour, weekends and yearly price rises do affect fares — this is **feature engineering**
Compute **distance_km** with the Haversine formula	raw coordinates alone say little; distance is the strongest driver of fare
Keep fare > 0, distance > 0, passengers 1–6	negative fares, zero-distance trips and 200-passenger cars are data-entry errors
Remove outliers in fare and distance with the **IQR rule**	extreme values pull the regression line towards themselves

The Haversine formula

The Earth is a sphere, so straight-line (Euclidean) distance on latitude/longitude is wrong. Haversine gives the **great-circle distance**:

$$a = \sin^2(\Delta\varphi/2) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2(\Delta\lambda/2)$$
$$d = 2R \cdot \arcsin(\sqrt{a}),\ R = 6371\ \text{km}$$

φ = latitude, λ = longitude, both converted to **radians** first (`np.radians`).

Outliers and the IQR rule

An **outlier** is a value far from the rest of the data. **Q1** = 25th percentile, **Q3** = 75th percentile, **IQR = Q3 − Q1**. Any value below **Q1 − 1.5·IQR** or above **Q3 + 1.5·IQR** is an outlier — these are the whiskers of a **box plot**.

Example: if Q1 = ₹6 and Q3 = ₹12.5, IQR = 6.5, so valid fares lie between 6 − 9.75 = −3.75 and 12.5 + 9.75 = 22.25. A ₹250 fare is removed.

Correlation

Pearson correlation r measures the strength of a *linear* relationship between two variables, from −1 (perfect negative) through 0 (none) to +1 (perfect positive). The code prints each feature's correlation with `fare_amount`; **distance_km** comes out highest by far, confirming it as the key feature. Correlation does not imply causation, and r near 0 only means no *linear* relation.

Step 2 — The two models

Linear Regression

Fits a straight line (a hyperplane, with many features):

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \ldots + b_nx_n$$

The coefficients are chosen by **Ordinary Least Squares (OLS)** — minimise the sum of squared errors $\Sigma(y_i - \hat{y}_i)^2$. Each b_i means "how much the fare changes when x_i increases by 1, others fixed". It is fast and interpretable but can only capture straight-line relationships.

Assumptions: linear relationship, independent errors, constant error variance (**homoscedasticity**), normally distributed errors, and no strong correlation among features (**no multicollinearity**).

Random Forest Regressor

A **decision tree** splits data with if-else questions ("distance < 3.2 km?") and predicts the average fare in each leaf. One deep tree overfits. A **random forest** builds many trees and **averages** them:

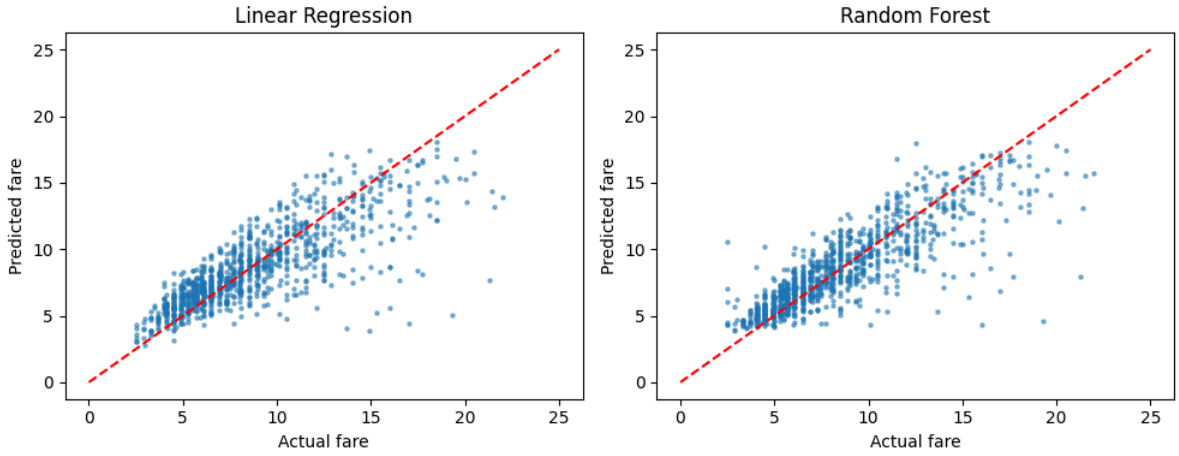
1. **Bagging (bootstrap aggregation)** — each tree is trained on a random sample of rows drawn *with replacement*.
2. **Feature randomness** — each split considers only a random subset of features, so trees differ from each other.
3. **Aggregation** — regression: average the trees' predictions; classification: majority vote.

Averaging many different, high-variance trees cancels out their individual errors, so the forest has **lower variance** and handles **non-linear** patterns and feature interactions (e.g. a short trip at 6 pm costing more than at 3 am). In the code: `n_estimators=50` trees, `max_depth=15` (limits overfitting), `n_jobs=-1` (use all CPU cores).

Step 3 — Evaluation metrics

Metric	Formula	Meaning	Better
MAE	$(1/n)\ \Sigma\ \lvert y - \hat{y}\rvert$	average error in ₹/$	lower
MSE	$(1/n)\ \Sigma\ (y - \hat{y})^2$	squares errors → punishes big mistakes	lower
RMSE	√MSE	same unit as fare, sensitive to large errors	lower
R^2	1 − SS_res / SS_tot	share of variance explained; 1 = perfect, 0 = as good as predicting the mean, can be negative	higher

Worked example: actual fares [10, 12, 8], predicted [11, 12, 6]. Errors 1, 0, −2. MAE = (1+0+2)/3 = **1.0**; MSE = (1+0+4)/3 = 1.67; RMSE = **1.29**. Mean fare = 10, SS_tot = 0 + 4 + 4 = 8, SS_res = 5 → R^2 = 1 − 5/8 = **0.375**.



Your program's plot: actual vs predicted fare. Points on the red line are perfect predictions. Linear regression under-predicts long expensive rides; the forest follows them a little better.

Conclusion to write: Random Forest usually gives a higher R^2 and lower RMSE/MAE than Linear Regression because fare depends on distance, time and location in non-linear ways. Linear Regression remains useful as a simple, explainable baseline.

Probable viva questions — Practical 8

Q1. What type of ML problem is fare prediction?

Supervised learning — regression, because the target (fare) is a continuous number and we have past rides with known fares.

Q2. Which pre-processing steps did you perform?

Dropped ID columns, removed missing rows, extracted hour/weekday/month/year from the datetime, computed trip distance with Haversine, removed impossible values (fare ≤ 0, distance 0, passengers outside 1–6), and removed outliers in fare and distance with the IQR rule.

Q3. What is an outlier? How did you detect it?

A value that lies unusually far from the rest. Detected with the IQR (box-plot) rule: values below Q1 − 1.5·IQR or above Q3 + 1.5·IQR. Box plots and z-scores ($|z| > 3$) are other methods.

Q4. Why remove outliers before linear regression?

OLS minimises squared errors, so one extreme point produces a huge squared error and drags the line towards itself, distorting predictions for normal rides.

Q5. Why use the Haversine formula instead of Euclidean distance?

Latitude and longitude are angles on a sphere, not points on a flat plane. Haversine computes the great-circle distance on the Earth's surface in kilometres.

Q6. What is feature engineering? Give examples from this practical.

Creating new, more useful features from raw data. Here: distance_km from four coordinate columns, and hour, weekday, month and year from the pickup timestamp.

Q7. What is correlation? Which feature correlated most with fare?

A number from −1 to +1 showing how strongly two variables move together linearly. distance_km has the highest correlation with fare_amount.

Q8. Explain linear regression.

It models the target as a weighted sum of features plus an intercept, $\hat{y} = b_o + \Sigma b_i x_i$, and finds the weights that minimise the sum of squared errors (ordinary least squares).

Q9. What are the assumptions of linear regression?

Linearity, independence of errors, homoscedasticity (constant error variance), normally distributed errors, and no multicollinearity among features.

Q10. What is multicollinearity?

When two or more features are strongly correlated with each other (e.g. pickup and drop-off latitude). Coefficients become unstable and hard to interpret, although predictions may still be fine.

Q11. Explain Random Forest.

An ensemble of decision trees, each trained on a bootstrap sample of rows and using a random subset of features at each split. For regression the trees' predictions are averaged. This reduces the overfitting of a single tree.

Q12. What is bagging? How is it different from boosting?

Bagging trains models independently and in parallel on bootstrap samples and averages them (reduces variance) — Random Forest. Boosting trains models sequentially, each correcting the previous one's errors (reduces bias) — AdaBoost, Gradient Boosting, XGBoost.

Q13. Why does Random Forest usually beat Linear Regression here?

The fare depends on distance, time of day and location in non-linear and interacting ways. Trees capture these patterns; a straight line cannot.

Q14. What do `n_estimators` and `max_depth` control?

`n_estimators` = number of trees (more = more stable, slower). `max_depth` = maximum depth of each tree; limiting it prevents trees from memorising noise.

Q15. Explain R^2. Can it be negative?

R^2 = 1 − (sum of squared residuals / total sum of squares) — the fraction of the target's variance explained by the model. 1 is perfect, 0 means no better than always predicting the mean, and it is negative when the model is worse than that.

Q16. Difference between MAE, MSE and RMSE?

MAE averages absolute errors and treats all errors equally. MSE averages squared errors, so large errors count much more. RMSE is the square root of MSE, bringing it back to the target's unit (dollars). RMSE ≥ MAE always.

Q17. Which metric would you report to a manager and why?

RMSE or MAE, because they are in money: "on average our prediction is off by about $X". R^2 is useful for comparing models but harder to explain.

Q18. Do decision trees or random forests need feature scaling?

No. Trees split on thresholds of one feature at a time, so the scale of a feature does not change the splits. Linear regression with regularisation, KNN, SVM and neural networks do benefit from scaling.

Q19. What does `train_test_split(..., test_size=0.2, random_state=42)` do?

Shuffles the data and puts 80% in the training set and 20% in the test set; random_state = 42 makes the shuffle identical every run.

Q20. Why does the code use `matplotlib.use("Agg")`?

The Agg backend draws plots to files without opening a window, so the script also runs on machines without a display.

PRACTICAL 9

Email spam detection — KNN and SVM

Aim. Classify emails as Spam or Not Spam (binary classification) using K-Nearest Neighbours and Support Vector Machine, and analyse their performance.

Code: `ML_EmailSpamKNN_SVM_pract9.py` . **Dataset:** `emails.csv` — about 5,170 emails × 3,002 columns: `Email No.` , 3,000 word-count columns and `Prediction` (1 = spam, 0 = not spam). Roughly 29% of the emails are spam.

How text becomes numbers — Bag of Words

Each column is one common word and each cell is how many times that word appears in that email. Word order is ignored, hence "bag of words". The dataset is already in this form, so no text processing is needed. The result is a **high-dimensional, sparse** matrix — most counts are 0.

`normalize(X)` scales each email (row) to unit length (L2 norm = 1). A long email with every word repeated twice would otherwise look very different from a short one with the same content. After normalisation the comparison is about *proportions* of words — it makes Euclidean distance behave like cosine similarity.

K-Nearest Neighbours (KNN)

Idea: similar things are close together. To classify a new email:

1. Compute its distance to **every** training email.
2. Take the **k** closest ones (here k = 5).
3. **Majority vote** of their labels is the prediction.

KNN is a **lazy learner** (no training phase — `fit` only stores the data) and **non-parametric** (no fixed set of learned weights). All the cost is at prediction time: O(n·d) per query for n training rows and d features.

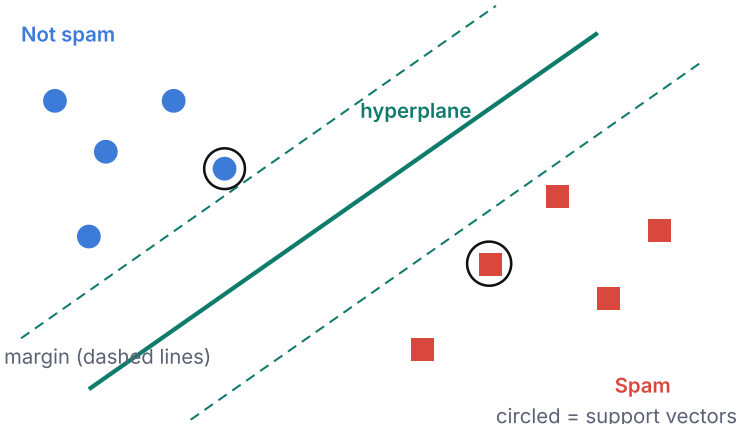
Distance measures: Euclidean $\sqrt{\Sigma(x_i - y_i)^2}$, Manhattan $\Sigma|x_i - y_i|$, Minkowski (general form), cosine similarity (common for text).

Choosing k: small k (1) follows noise → overfitting; large k smooths too much → underfitting. Use an **odd k** for two classes to avoid ties, and pick it with cross-validation (Practical 11 does this).

Weakness here — curse of dimensionality: with 3,000 dimensions, all points are roughly equally far apart, so "nearest" becomes less meaningful.

Support Vector Machine (SVM)

SVM finds the **decision boundary (hyperplane)** that separates the two classes with the **maximum margin** — the widest possible gap between the boundary and the closest points of each class. Those closest points are the **support vectors**; only they determine the boundary.



SVM picks the separating line with the widest margin; only the circled support vectors decide where it goes.

Soft margin and C. Real data is not perfectly separable. The parameter **C** controls the penalty for points on the wrong side: large C → narrow margin, few training errors (risk of overfitting); small C → wider margin, more tolerance (smoother). The code uses C = 10.

Kernel trick. When classes are not linearly separable, a kernel computes similarity *as if* the data were mapped to a higher-dimensional space where a linear separator exists — without actually computing that mapping. Common kernels: **linear**, **polynomial**, **RBF / Gaussian** $K(x, y) = \exp(-\gamma\|x - y\|^2)$. The code uses RBF; γ controls how far one training point's influence reaches.

Evaluating a classifier

	Predicted Not spam (0)	Predicted Spam (1)
Actual Not spam (0)	TN — correct	FP — good mail sent to spam ×
Actual Spam (1)	FN — spam reached inbox ×	TP — correct

scikit-learn prints the confusion matrix as `[[TN FP] [FN TP]]` (rows = actual, columns = predicted).

Metric	Formula	Question it answers
Accuracy	(TP + TN) / total	how many predictions were right?
Error rate	(FP + FN) / total = 1 − accuracy	how many were wrong?
Precision	TP / (TP + FP)	of mails flagged spam, how many really were?
Recall (sensitivity)	TP / (TP + FN)	of all real spam, how much did we catch?
F1-score	2·P·R / (P + R)	balance of precision and recall
Specificity	TN / (TN + FP)	of good mails, how many did we keep?

For spam filters **precision** is especially important — a false positive hides a genuine (maybe important) email, which is worse than letting one spam through.

Result to expect: SVM generally beats KNN on this dataset (higher accuracy and F1), because SVM handles high-dimensional sparse data well while KNN's distances degrade. KNN is also much slower at prediction time.

Probable viva questions — Practical 9

Q1. What is binary classification? What are the classes here?

Predicting one of exactly two classes. Here 0 = Not Spam (normal state) and 1 = Spam (abnormal state).

Q2. How is an email represented in this dataset?

As a bag-of-words vector: 3,000 columns, each the count of one word in that email. Word order is ignored.

Q3. Explain the KNN algorithm.

Store all training data. For a new point, compute its distance to every training point, pick the k nearest, and predict the majority class among them (or the average, for regression).

Q4. Why is KNN called a lazy learner?

It does no learning during `fit` — it just stores the data. All computation (distances, sorting) happens at prediction time.

Q5. Is KNN parametric or non-parametric?

Non-parametric — it does not assume a fixed form or learn a fixed number of parameters; the "model" is the training data itself.

Q6. How do you choose the value of k?

Try a range of k with cross-validation and pick the best. Use an odd k for binary classes to avoid ties. A common rule of thumb starts near √n.

Q7. What happens if k is too small or too large?

k = 1: very sensitive to noise and outliers (overfitting, high variance). Very large k: the majority class dominates and local patterns are lost (underfitting, high bias).

Q8. What distance metrics can KNN use?

Euclidean (default), Manhattan, Minkowski (generalises both), Chebyshev, Hamming (for categorical/binary), and cosine similarity (common for text).

Q9. What is the curse of dimensionality?

As the number of features grows, data becomes sparse and the distances between all pairs of points become similar, so nearest-neighbour methods lose their meaning and need exponentially more data.

Q10. Explain SVM.

SVM finds the hyperplane that separates the classes with the largest margin. The margin is defined by the nearest points of each class — the support vectors. Kernels let it learn non-linear boundaries.

Q11. What are support vectors?

The training points closest to the decision boundary. They alone define the boundary — removing any other point would not change it.

Q12. What is the margin and why maximise it?

The distance between the hyperplane and the nearest points of either class. A wider margin means the classifier is less sensitive to small changes in the data, so it generalises better.

Q13. What is the kernel trick?

Computing the dot product of points in a high-dimensional feature space directly from the original features through a kernel function, so SVM can find non-linear boundaries without explicitly transforming the data.

Q14. Name the SVM kernels. Which did you use?

Linear, polynomial, RBF (Gaussian) and sigmoid. The code uses RBF (`SVC(kernel="rbf", C=10)`).

Q15. What is the role of C in SVM?

C is the penalty for misclassified training points. High C → smaller margin and fewer training errors (may overfit). Low C → larger margin, allows some errors (more regularised).

Q16. What is gamma in the RBF kernel?

It controls how far the influence of one training point reaches. High gamma → each point affects only its close neighbourhood (wiggly boundary, overfitting). Low gamma → smoother boundary.

Q17. Why did SVM perform better than KNN?

Text data has thousands of sparse dimensions. SVM depends only on support vectors and works well in high dimensions; KNN's distance comparisons become unreliable there and it is sensitive to irrelevant features.

Q18. Why was `normalize` used?

To scale every email to unit length so that long emails do not appear far from short ones just because they have bigger counts. It makes the comparison depend on word proportions.

Q19. Explain the confusion matrix.

A 2 × 2 table of actual vs predicted classes: TN, FP in the first row (actual 0) and FN, TP in the second row (actual 1). Every metric — accuracy, precision, recall, F1 — is computed from it.

Q20. Precision vs recall — which matters more for spam?

Usually precision: a false positive moves a genuine email to the spam folder, which can be costly. Recall matters more when missing a positive is dangerous (disease detection).

Q21. Why is accuracy alone not enough?

With imbalanced classes a model can score high accuracy by always predicting the majority class. Precision, recall and F1 show how well the minority class is handled.

Q22. What is F1-score?

The harmonic mean of precision and recall, 2PR / (P + R). It is high only when both are high.

Q23. What does `stratify=y` do in train_test_split?

It keeps the same proportion of spam and non-spam emails in both the training and the test sets.

PRACTICAL 10

Bank customer churn — Neural network classifier

Aim. Build a neural-network classifier that predicts whether a bank customer will leave in the next 6 months: read the data, separate features and target, normalise, train/test split, build the model, and report accuracy and the confusion matrix with points for improvement.

Code: `ML_BankChurnNeuralNetwork_pract10.py`. **Dataset:** `Churn_Modelling.csv` — 10,000 customers with CreditScore, Geography, Gender, Age, Tenure, Balance, NumOfProducts, HasCrCard, IsActiveMember, EstimatedSalary and the target **Exited** (1 = left). About **20% of customers left** — the classes are imbalanced.

Data preparation

Step	Code	Reason
Drop RowNumber, CustomerId, Surname	`df.drop(...)`	identifiers carry no information about leaving; keeping Surname could even cause overfitting
Encode Gender	`map({"Female": 0, "Male": 1})`	**label encoding** — two categories become 0/1
One-hot encode Geography	`pd.get_dummies(...)`	France/Germany/Spain become three 0/1 columns. Label-encoding them 0, 1, 2 would falsely imply Spain > Germany > France
Split 80/20, stratified	`train_test_split(..., stratify=y)`	keeps the 20% churn ratio in both sets
Standardise	`StandardScaler` — fit on train, transform test	Balance (up to ~250,000) and Tenure (0–10) need equal scales for gradient descent to converge well

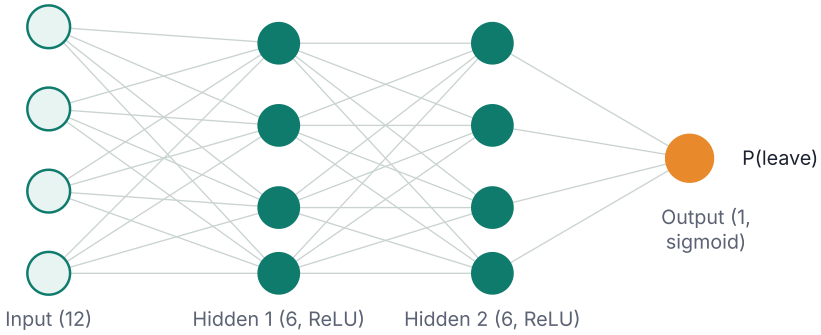
After encoding there are **12 input features**.

Artificial neural network basics

A neuron computes a weighted sum of its inputs plus a bias, then applies an activation function:

$$z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b \rightarrow \text{output } a = f(z)$$

Layers: input layer (one node per feature) → hidden layers (learn patterns) → output layer. `Dense` means **fully connected** — every neuron is connected to every neuron in the next layer.



The basic 12 → 6 → 6 → 1 network (fewer circles drawn than real neurons).

Activation functions

Function	Formula	Range	Used where
ReLU	max(0, z)	0 to ∞	hidden layers — simple, fast, avoids vanishing gradients
Sigmoid	$1 / (1 + e^{-z})$	0 to 1	output for binary classification → probability
Tanh	$(e^{z} - e^{-z})/(e^{z} + e^{-z})$	−1 to 1	hidden layers in older / recurrent nets
Softmax	$e^{z_i} / \Sigma e^{z_j}$	0 to 1, sums to 1	output for multi-class

Without non-linear activations, any number of layers would collapse into one linear function.

How the network learns

1. **Forward propagation** — inputs flow through the layers to give a predicted probability ŷ.
2. **Loss — binary cross-entropy**: $L = -[y \cdot \log \hat{y} + (1 - y) \cdot \log(1 - \hat{y})]$. It heavily punishes confident wrong predictions.
3. **Backpropagation** — the chain rule computes how much each weight contributed to the loss (the gradient $\partial L/\partial w$), from the output layer back to the input.
4. **Update** — the optimiser moves each weight against its gradient: $w \leftarrow w - \eta \cdot \partial L/\partial w$ (η = learning rate). **Adam** adapts the learning rate for each weight using running averages of gradients (momentum + RMSProp).

Epoch = one full pass over the training data. **Batch size** = number of samples processed before one weight update (32 here, so 8,000 / 32 = 250 updates per epoch).

Prediction: `model.predict` returns a probability; the code converts it to a class with **threshold 0.5** (`> 0.5` → leaves).

Trainable parameters

Each Dense layer has (inputs × neurons) weights + neurons biases. **Basic model:** 12×6+6 = 78, 6×6+6 = 42, 6×1+1 = 7 → **127 parameters**. **Improved model:** 12×32+32 = 416, 32×16+16 = 528, 16×1+1 = 17 → **961 parameters** (Dropout adds none).

Why the basic model is not enough, and the improvements

Because only ~20% of customers leave, a model that predicts "stays" for everyone already gets ~80% accuracy. The basic network typically reaches around 85% accuracy but has **low recall for the "Left" class** — it misses many churners, which is exactly the group the bank cares about.

Improvement in the code	What it does
More neurons (32 → 16)	more capacity to learn patterns
Dropout(0.2)	randomly switches off 20% of neurons in each training step, so the network cannot rely on a few neurons — reduces overfitting. Disabled automatically at prediction time
EarlyStopping(patience=10, restore_best_weights=True)	watches validation loss and stops when it has not improved for 10 epochs, then restores the best weights
validation_split=0.2	holds back 20% of training data to monitor overfitting during training
class_weight={0: 1, 1: 2}	a mistake on a churner costs twice as much in the loss, pushing the model to catch more churners — recall for "Left" rises, precision may fall slightly

Other ideas you can mention: tune the threshold (e.g. 0.4), oversample the minority class (SMOTE), batch normalisation, tune learning rate / layers with a grid search, or compare with Random Forest / XGBoost.

Probable viva questions — Practical 10

Q1. What is customer churn?

When a customer stops doing business with a company — here, closing the bank account. Predicting it lets the bank offer retention deals before the customer leaves.

Q2. Which columns did you drop and why?

RowNumber, CustomerId and Surname — they are identifiers with no causal link to leaving, and they can cause the model to memorise individuals.

Q3. Why one-hot encode Geography but label-encode Gender?

Gender has two values, so 0/1 is enough. Geography has three unordered values; encoding them as 0/1/2 would invent a false order and distance, so each becomes its own 0/1 column.

Q4. What is the dummy variable trap?

With one-hot columns for every category, one column can be predicted exactly from the others (they sum to 1), causing perfect multicollinearity in linear models. Dropping one column (`drop_first=True`) avoids it; neural networks are much less affected.

Q5. Why normalise the data for a neural network?

Gradient descent converges faster and more reliably when features have similar scales. Otherwise large-valued features like Balance dominate the weighted sums and gradients.

Q6. Why is the scaler fitted only on the training data?

Fitting it on the full data would leak the test set's mean and standard deviation into training, giving optimistic results. The test set must be transformed with the training statistics.

Q7. What is an artificial neural network?

A model made of layers of connected neurons. Each neuron computes a weighted sum of its inputs plus a bias and passes it through an activation function. Weights are learned from data using backpropagation and gradient descent.

Q8. What is a perceptron?

The simplest neural network — a single neuron with a step activation that outputs 0 or 1. It can only learn linearly separable problems (it cannot learn XOR); stacking neurons in layers (multi-layer perceptron) removes this limit.

Q9. Why do we need activation functions?

To introduce non-linearity. Without them, stacking layers is still just one linear transformation and the network could not learn complex patterns.

Q10. Why ReLU in hidden layers and sigmoid in the output?

ReLU is cheap and does not saturate for positive inputs, which helps avoid vanishing gradients in hidden layers. Sigmoid squashes the output to 0–1, which we interpret as the probability of leaving.

Q11. What is the loss function and why that one?

Binary cross-entropy, the standard loss for a two-class problem with a probability output. It strongly penalises confident wrong predictions and works well with sigmoid outputs.

Q12. Explain backpropagation.

After a forward pass computes the loss, backpropagation applies the chain rule from the output layer backwards to compute the gradient of the loss with respect to every weight and bias. The optimiser then updates the weights using these gradients.

Q13. What is gradient descent? What is the learning rate?

An optimisation method that repeatedly moves the weights a small step in the direction that reduces the loss most (negative gradient). The learning rate is the step size — too large overshoots, too small learns very slowly.

Q14. What is the Adam optimiser?

Adaptive Moment Estimation — gradient descent that keeps a running average of past gradients (momentum) and of their squares, giving each weight its own adaptive learning rate. It usually converges fast with little tuning.

Q15. Difference between epoch, batch and iteration?

Epoch: one full pass through the training data. Batch: a subset of samples used for one update (32 here). Iteration: one weight update. With 8,000 samples and batch size 32 there are 250 iterations per epoch.

Q16. How many trainable parameters does the basic model have?

127: $(12\times6 + 6) + (6\times6 + 6) + (6\times1 + 1) = 78 + 42 + 7$.

Q17. Why can accuracy be misleading for this dataset?

About 80% of customers stay, so predicting "stays" for everyone gives ~80% accuracy while finding zero churners. Recall, precision and F1 for the "Left" class show the real picture.

Q18. What is Dropout?

A regularisation technique that randomly sets a fraction of neurons' outputs to zero during each training step. The network cannot depend on specific neurons, so it generalises better. At prediction time all neurons are used.

Q19. What is Early Stopping?

A callback that monitors validation loss each epoch and stops training once it stops improving for a set number of epochs (patience), restoring the best weights. It prevents overfitting and saves time.

Q20. What are class weights and why use them here?

Weights that scale the loss for each class. Setting class 1 to 2 makes each missed churner count double, so the model pays more attention to the minority class and its recall improves.

Q21. How do you convert the sigmoid output into a class?

Compare it with a threshold, by default 0.5: probability > 0.5 → "Left" (1), otherwise "Stayed" (0). Lowering the threshold increases recall at the cost of precision.

Q22. What is the vanishing gradient problem?

In deep networks with sigmoid/tanh activations, gradients become extremely small as they are multiplied backwards through layers, so early layers stop learning. ReLU, good initialisation and batch normalisation help.

Q23. How could the model be improved further?

Tune the threshold, oversample churners (SMOTE), tune the architecture and learning rate, add batch normalisation, use k-fold cross-validation, or compare with gradient-boosting models.

PRACTICAL 11

K-Nearest Neighbours on the diabetes dataset

Aim. Implement KNN on `diabetes.csv` and compute the confusion matrix, accuracy, error rate, precision and recall.

Code: `ML_DiabetesKNN_pract11.py`. **Dataset:** Pima Indians Diabetes — 768 female patients; features Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age; target **Outcome** (1 = diabetic, about 35% of patients).

Pre-processing done in the code

1. **Zeros as missing values.** Glucose, BloodPressure, SkinThickness, Insulin and BMI cannot be 0 in a living person. The dataset uses 0 to mean "not measured", so they are replaced with NaN.
2. **Outlier count** with the IQR rule, per column (reported, not removed).
3. **Correlation with Outcome** — Glucose has the strongest correlation; BMI, Insulin and Age are also positively correlated.
4. **Stratified 80/20 split.**
5. **Median imputation** — missing values are filled with the **median of the training set**, and the same medians are used for the test set (no leakage). Median is preferred over mean because it is not pulled by outliers (Insulin is heavily skewed).
6. **Scaling inside a Pipeline** — `make_pipeline(StandardScaler(), KNeighborsClassifier())`. Inside cross-validation the scaler is re-fitted on each training fold only.

KNN worked example

Training points (2 features): A: (1,1), (2,1), (1,2) and B: (5,5), (6,5), (5,6). New point **(2, 2)**, k = 3.

Training point	Class	Euclidean distance to (2, 2)
(2, 1)	A	$\sqrt{(0^2 + 1^2)} = 1.000$
(1, 2)	A	$\sqrt{(1^2 + 0^2)} = 1.000$
(1, 1)	A	$\sqrt{(1^2 + 1^2)} = 1.414$
(5, 5)	B	$\sqrt{(3^2 + 3^2)} = 4.243$
(6, 5)	B	$\sqrt{(4^2 + 3^2)} = 5.000$
(5, 6)	B	$\sqrt{(3^2 + 4^2)} = 5.000$

The 3 nearest are all class A → prediction **A**.

Why scaling matters for KNN: if one feature is Insulin (up to ~800) and another is DiabetesPedigreeFunction (0–2.4), the distance is almost entirely decided by Insulin. After standardisation every feature contributes fairly.

Choosing k with GridSearchCV

`GridSearchCV` tries every k from 1 to 25 with **5-fold cross-validation** on the training data: the training data is split into 5 parts, each k is trained on 4 parts and scored on the 5th, rotated 5 times, and the average accuracy is kept. The k with the best average is chosen and the model is refitted on all training data. The test set is touched only once, at the end.

Metrics from the confusion matrix

	Predicted 0 (healthy)	Predicted 1 (diabetic)
Actual 0	TN	FP — false alarm
Actual 1	FN — missed patient	TP

Worked example — suppose TN = 85, FP = 15, FN = 14, TP = 40 (154 test patients):

- Accuracy = (40 + 85) / 154 = **0.812**
- Error rate = (15 + 14) / 154 = **0.188** = 1 − accuracy
- Precision = 40 / (40 + 15) = **0.727**
- Recall = 40 / (40 + 14) = **0.741**
- F1 = 2 × 0.727 × 0.741 / (0.727 + 0.741) = **0.734**

In medical screening **recall is most important** — a false negative sends a diabetic patient home untreated, while a false positive only leads to an extra test.

Probable viva questions — Practical 11

Q1. Explain KNN in simple words.

To classify a new patient, find the k patients in the training data most similar to them (smallest distance) and predict the class most of them have.

Q2. Why did you replace zeros with NaN?

A glucose, blood pressure, skin thickness, insulin or BMI value of 0 is medically impossible; in this dataset 0 means "missing". Treating it as a real value would distort distances and statistics.

Q3. Why fill missing values with the median and not the mean?

The median is robust to outliers and skewed data (Insulin has a long right tail). The mean would be pulled towards extreme values.

Q4. Why are the medians computed from the training data only?

Using test-set statistics during training is data leakage. In real use, future patients' data is not available at training time.

Q5. What is imputation? Name some methods.

Filling missing values. Methods: mean, median, mode, constant, forward/backward fill, KNN imputation, model-based (regression) imputation — or simply dropping rows.

Q6. Why is feature scaling important for KNN?

KNN relies on distance; features with large numeric ranges would dominate the distance and the small-range features would be ignored.

Q7. What is a Pipeline and why use it here?

A Pipeline chains steps (scaler → classifier) into one object. With cross-validation it re-fits the scaler on each training fold only, preventing leakage, and keeps the code simple.

Q8. What is cross-validation? Why 5-fold?

Splitting the training data into k folds, training on k − 1 and validating on the remaining fold, rotating through all folds and averaging. It gives a more reliable estimate than one split. 5 or 10 folds is a common balance between reliability and computation.

Q9. What does GridSearchCV do?

It tries every combination of the given hyperparameter values (here k = 1...25), evaluates each with cross-validation, and keeps the best one (`best_params_`). The final model is refitted on all training data.

Q10. Define TP, TN, FP and FN for this problem.

TP: diabetic predicted diabetic. TN: healthy predicted healthy. FP: healthy predicted diabetic (false alarm). FN: diabetic predicted healthy (missed case).

Q11. Give the formulas for accuracy, error rate, precision and recall.

Accuracy = (TP + TN)/total. Error rate = (FP + FN)/total = 1 − accuracy. Precision = TP/(TP + FP). Recall = TP/(TP + FN).

Q12. Which metric is most important for diabetes detection and why?

Recall (sensitivity) — missing a diabetic patient (FN) is more harmful than a false alarm, which can be checked with a follow-up test.

Q13. What is the difference between sensitivity and specificity?

Sensitivity (recall) = TP/(TP + FN), how well we find positives. Specificity = TN/(TN + FP), how well we identify negatives.

Q14. Which feature is most correlated with diabetes in this dataset?

Glucose, clearly ahead of the others. BMI, Insulin and Age are also positively correlated with the outcome.

Q15. What are the advantages and disadvantages of KNN?

Advantages: simple, no training, naturally multi-class, no assumptions about data distribution. Disadvantages: slow prediction on large data, needs scaling, sensitive to irrelevant features and noise, suffers from the curse of dimensionality, needs memory for all training data.

Q16. Can KNN be used for regression?

Yes — predict the average (or distance-weighted average) of the k nearest neighbours' target values.

Q17. What is weighted KNN?

Neighbours vote with weight proportional to 1/distance, so closer neighbours have more influence (`weights="distance"` in scikit-learn).

Q18. What is the time complexity of KNN?

Training O(1) (just stores data). Prediction O(n·d) per query for n training points and d features. KD-trees or ball trees speed this up in low dimensions.

Q19. Why stratify the split?

Only about 35% of patients are diabetic; stratifying keeps that ratio in both training and test sets so the evaluation is representative.

PRACTICAL 12

K-Means and Hierarchical clustering — Mall customers

Aim. Implement K-Means (and hierarchical) clustering on the customer dataset and determine the number of clusters with the elbow method.

Code: `ML_KMeansClustering_pract12.py`. **Dataset:** `Mall_Customers.csv` — 200 customers with Gender, Age, Annual Income (k$) and Spending Score (1–100). Clustering uses **Annual Income** and **Spending Score**.

Clustering

Clustering is **unsupervised learning**: there are no labels; the algorithm groups points so that points in the same cluster are similar and points in different clusters are different. Business use: **customer segmentation** — target each group with a different marketing strategy.

K-Means algorithm

1. Choose **k** (number of clusters).
2. **Initialise** k centroids. The code uses **k-means++**, which spreads the initial centroids out (each new one chosen with probability proportional to its squared distance from existing ones) — faster convergence and better results than random initialisation.
3. **Assign** every point to its nearest centroid (Euclidean distance).
4. **Update** each centroid to the mean of the points assigned to it.
5. **Repeat** 3–4 until assignments stop changing (convergence) or a maximum number of iterations.

K-Means minimises **WCSS (within-cluster sum of squares)**, also called **inertia**: the sum of squared distances of each point from its cluster's centroid. `n_init=10` runs the whole algorithm 10 times with different starts and keeps the run with the lowest WCSS, because K-Means can get stuck in a local minimum.

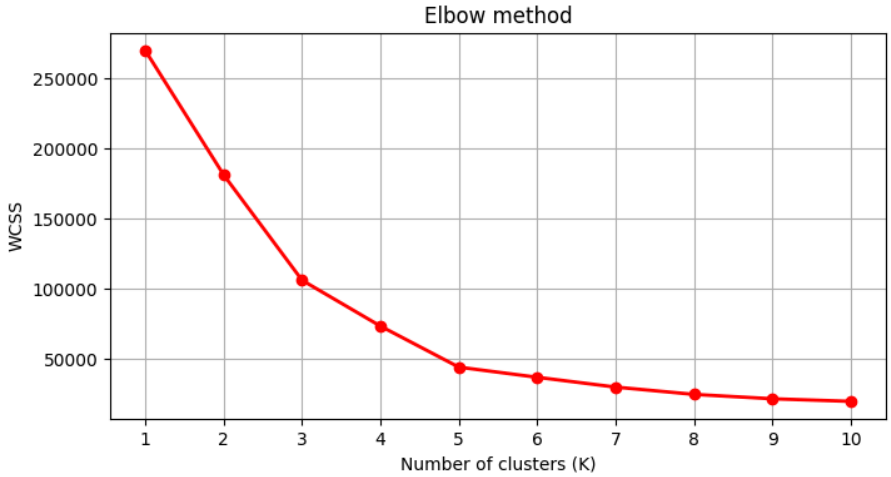
Worked iteration (k = 2)

Points: (1,1), (1.5,2), (3,4), (5,7), (3.5,5), (4.5,5), (3.5,4.5). Initial centroids C1 = (1,1), C2 = (5,7).

Iteration	Cluster 1	Cluster 2	New centroids
1	(1,1), (1.5,2), (3,4)	(5,7), (3.5,5), (4.5,5), (3.5,4.5)	C1 = (1.83, 2.33), C2 = (4.13, 5.38)
2	(1,1), (1.5,2)	(3,4) moves here + the other four	C1 = (1.25, 1.5), C2 = (3.9, 5.1)
3	no change	no change	**converged**

Choosing k — the elbow method

Run K-Means for k = 1…10 and plot WCSS. WCSS always falls as k grows (with k = n it is 0), so we look for the **elbow** — the k after which adding clusters gives only a small improvement.

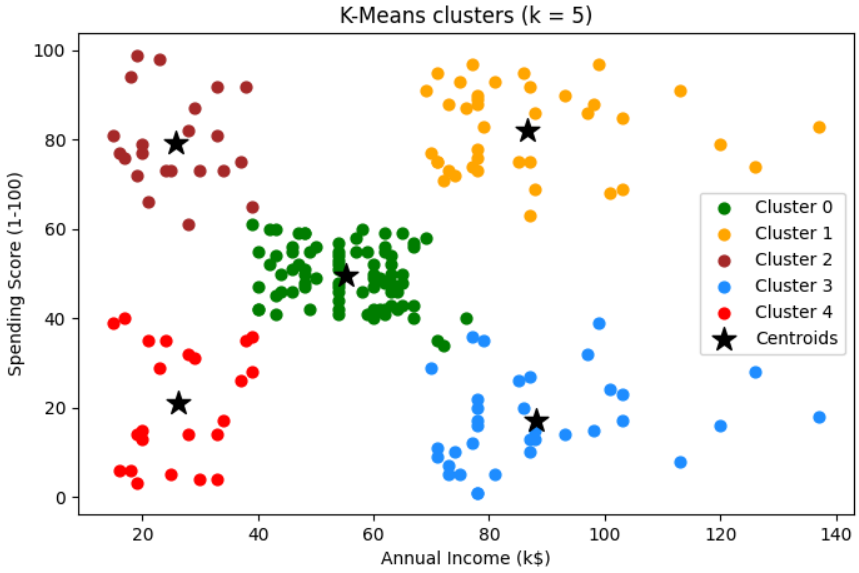


Your elbow plot. The drop is large up to k = 5 and small afterwards, so k = 5.

The code picks the elbow automatically: the first k where going to k + 1 reduces WCSS by less than 10% of the very first drop (k = 1 → 2). That gives **k = 5**.

Silhouette score confirms the choice. For each point, a = average distance to points in its own cluster, b = average distance to points in the nearest other cluster, s = (b − a) / max(a, b). It ranges from −1 to 1; close to 1 means well-separated clusters. The overall score is the mean over all points (about 0.55 for k = 5 here).

Interpreting the 5 clusters



Your K-Means output with k = 5; black stars are centroids.

Cluster (as in your plot)	Income	Spending	Segment and strategy
0 (green, centre)	medium	medium	"standard" customers — regular offers
1 (orange)	high	high	**target group** — premium products, loyalty programme
2 (brown)	low	high	"careless" spenders — discounts, trendy items
3 (blue)	high	low	"careful" rich — high potential; personalised marketing to win them over
4 (red)	low	low	"sensible" — budget offers, low priority

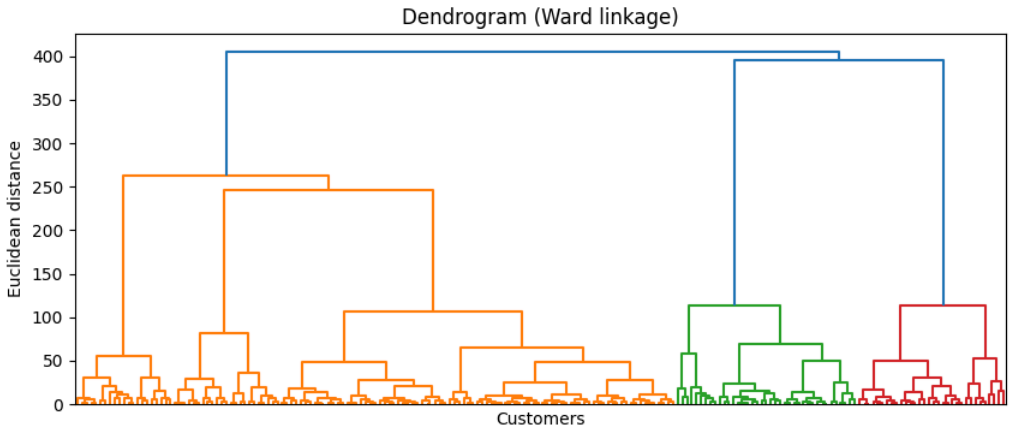
(Cluster numbers can change between runs; describe them by their position, not their number.)

Hierarchical clustering

Agglomerative (bottom-up): start with every point as its own cluster; repeatedly merge the two closest clusters until one remains. **Divisive (top-down)** does the reverse. The history of merges is drawn as a **dendrogram**; cutting it with a horizontal line at a chosen height gives the clusters — a good cut is across the tallest vertical lines that no merge crosses.

Linkage defines the distance between two clusters:

Linkage	Distance between clusters
Single	closest pair of points (can create long "chains")
Complete	farthest pair of points
Average	average of all pairwise distances
Ward (used here)	merge the pair that causes the smallest increase in total within-cluster variance — gives compact clusters similar to K-Means



Your dendrogram (Ward linkage). A horizontal cut through the tall vertical lines gives 5 clusters.

	K-Means	Hierarchical (agglomerative)
k needed in advance?	yes	no — choose by cutting the dendrogram
Complexity	about O(n·k·iterations) — fast for large data	$O(n^2)$ memory, $O(n^2 \log n)$–$O(n^3)$ time — small data only
Result	depends on initial centroids	deterministic
Cluster shape	roughly spherical	depends on linkage
Output	flat partition	tree of nested clusters

Probable viva questions — Practical 12

Q1. What is clustering? Is it supervised?

Grouping data so that items in one group are more similar to each other than to items in other groups. It is unsupervised — no labels are used.

Q2. Explain the K-Means algorithm.

Choose k; initialise k centroids; assign each point to the nearest centroid; recompute each centroid as the mean of its points; repeat assignment and update until the clusters stop changing.

Q3. What does K-Means minimise?

The within-cluster sum of squares (WCSS / inertia) — the total squared distance between each point and the centroid of its cluster.

Q4. What is the elbow method?

Plot WCSS against k. WCSS decreases as k increases; the "elbow" where the curve bends and the decrease becomes small gives a good k. For this dataset it is 5.

Q5. Why can't we just pick the k with the lowest WCSS?

WCSS always decreases as k increases and reaches 0 when every point is its own cluster, which is useless. We want the point of diminishing returns.

Q6. What is the silhouette score?

For each point, s = (b − a)/max(a, b), where a is the mean distance to its own cluster and b to the nearest other cluster. Values near 1 mean well-separated clusters, near 0 overlapping clusters, negative means probably mis-assigned. Averaged over all points.

Q7. What is k-means++?

A smarter initialisation: the first centroid is random, each next centroid is chosen with probability proportional to its squared distance from the nearest existing centroid. Centroids start spread out, which gives faster and better convergence.

Q8. Why does the code use `n_init=10`?

K-Means can converge to a local minimum that depends on the starting centroids. Running it 10 times and keeping the result with the lowest WCSS gives a more reliable solution.

Q9. Does K-Means always give the same result?

Not necessarily — different initial centroids can lead to different local minima. Fixing `random_state` makes runs repeatable.

Q10. What are the limitations of K-Means?

k must be chosen in advance; sensitive to initialisation and to outliers (means are pulled by them); assumes roughly spherical, similar-sized clusters; needs feature scaling when features have different units; works only with numeric data.

Q11. Why weren't the features scaled in this practical?

Annual Income (15–137 k$) and Spending Score (1–100) have similar ranges, so neither dominates the distance. With very different units, scaling would be required.

Q12. Which features did you cluster on, and why?

Annual Income and Spending Score — two features give a clear 2-D plot and the most actionable business segments. The code also computes WCSS using Age as a third feature for comparison.

Q13. Interpret the five clusters.

High income–high spending (prime target), low income–high spending, high income–low spending (potential), low income–low spending, and an average middle group. Each gets a different marketing strategy.

Q14. What is hierarchical clustering? Name its types.

Clustering that builds a hierarchy of clusters. Agglomerative (bottom-up — start with single points and merge) and divisive (top-down — start with one cluster and split).

Q15. What is a dendrogram and how do you get clusters from it?

A tree diagram showing which clusters merged and at what distance (height). Draw a horizontal line across it; the number of vertical lines it cuts is the number of clusters.

Q16. What is Ward linkage?

A linkage rule that merges the two clusters whose union gives the smallest increase in total within-cluster variance. It produces compact, evenly sized clusters.

Q17. Compare K-Means and hierarchical clustering.

K-Means needs k beforehand, is fast and scales to large data, but depends on initialisation. Hierarchical needs no k up front, gives a full dendrogram and is deterministic, but is slow ($O(n^2)$ memory or more) and suitable only for small datasets.

Q18. Name some other clustering algorithms.

DBSCAN (density-based, finds arbitrary shapes and marks noise points; no k needed), Mean-Shift, Gaussian Mixture Models (soft clustering) and K-Medoids (uses actual points as centres, more robust to outliers).

Q19. What is the difference between classification and clustering?

Classification is supervised — it learns from labelled examples to predict known classes. Clustering is unsupervised — it discovers groups with no labels; the meaning of each group is decided afterwards by us.

REVISION

Quick revision and cross-practical questions

One-page summary

Practical	Task	Type	Algorithm(s)	Key metric(s)
8 Uber	fare prediction	supervised – regression	Linear Regression, Random Forest	R^2, RMSE, MAE
9 Spam	spam vs not spam	supervised – binary classification	KNN, SVM	accuracy, precision, recall, F1
10 Churn	leaves / stays	supervised – binary classification	ANN (Keras)	accuracy, confusion matrix, recall for "Left"
11 Diabetes	diabetic / not	supervised – binary classification	KNN + GridSearchCV	accuracy, error rate, precision, recall
12 Mall	customer segments	unsupervised – clustering	K-Means, Agglomerative	WCSS (elbow), silhouette

Mixed questions

Q1. Which of your practicals use KNN, and what did each one teach?

Practical 9 (spam) showed KNN struggling with 3,000 sparse dimensions compared with SVM. Practical 11 (diabetes) showed how to tune k with cross-validation and why scaling and imputation matter.

Q2. In which practicals is feature scaling essential and in which is it not needed?

Essential: KNN (9, 11), SVM (9), neural network (10), K-Means when units differ (12). Not needed: Random Forest (8) — tree splits are scale-invariant.

Q3. Which practicals have imbalanced data and how did you handle it?

Churn (~20% leave), diabetes (~35% diabetic) and spam (~29% spam). Handled with stratified splits, metrics beyond accuracy (precision, recall, F1), and class weights in the churn network.

Q4. Where did you use the IQR rule?

To remove outliers in fare and distance (Practical 8) and to count outliers in the diabetes columns (Practical 11).

Q5. KNN vs K-Means — what is the difference?

KNN is a supervised classifier — k is the number of neighbours that vote. K-Means is unsupervised clustering — k is the number of clusters. The only similarity is that both use distances.

Q6. Which model is the most interpretable and which the least?

Most: Linear Regression (each coefficient has a direct meaning) and KNN (you can show the neighbours). Least: the neural network and the RBF-kernel SVM (black boxes). Random Forest is in between (feature importances are available).

Q7. How would you deploy one of these models?

Save the trained model and the fitted scaler (e.g. with joblib or `model.save()` in Keras), load them in a small API (Flask/FastAPI), apply exactly the same pre-processing to incoming data, and return the prediction.